

Meta Muse: Capabilities, Architecture & Banking VA Applicability

As of Sep 25, 2026 (revised for mobile-only VA)

Overview

Muse is Meta Superintelligence Labs’ (MSL) family of proprietary AI products: a frontier model (Muse Spark), a consumer personal agent (Muse) built on it, and media models (Muse Image). The agent launched on September 8, 2026 and runs each user’s assistant inside a dedicated, security-hardened cloud VM.

Date	Release	What it is
Sep 8, 2026	Muse (personal agent)	Autonomous agent on a per-user cloud VM; iOS, Android, web, WhatsApp
Jul 9, 2026	Muse Spark 1.1 + Meta Model API	First paid, hosted API access to Muse Spark (US public preview)
Jul 7, 2026	Muse Image	Diffusion Transformer image model driven by Muse Spark reasoning
Apr 8, 2026	Muse Spark	Natively multimodal reasoning model; first of the Muse family

Muse Spark is closed-weights and cloud-only, a break from Meta’s open-weights Llama line. Meta’s Muse is unrelated to Google’s 2023 research model of the same name (a masked-transformer text-to-image model). This write-up covers Meta’s Muse, with the agent as the main focus.

Capabilities

Muse’s core capability is autonomous, multi-step task execution with bounded risk: it plans, uses tools and a browser, runs in the background, and pauses for human approval on consequential actions.

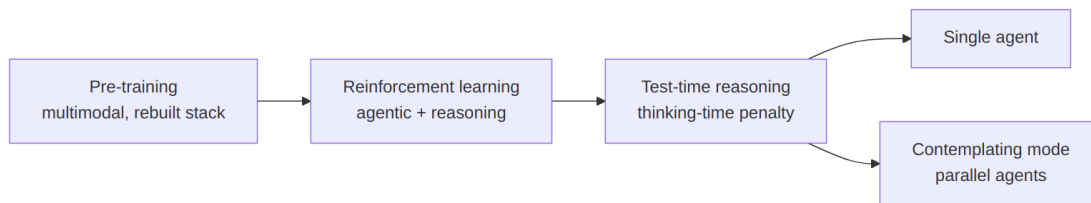
Layer	Capability	Detail
Model	Multimodal reasoning	Text, images, audio, video and PDFs in; visual chain-of-thought for image-based problems
Model	Tool calling	Zero-shot tool use via CLIs and skills; OpenAI-compatible function calling

Layer	Capability	Detail
Model	Long context	with parallel calls (API) Reported 1M-token context on Muse Spark 1.1
Model	Multi-agent reasoning	“Contemplating” mode runs parallel reasoning agents for hard problems
Model	Prompt-injection awareness	Trained to resist injected instructions in long trajectories
Agent	Background execution	Keeps working when the app is closed; cron jobs; reports at milestones
Agent	Sub-agent swarms	Launches concurrent sub-agents, including a dedicated browser sub-agent
Agent	Connectors + skills	Built-in connectors (email, calendar, Meta apps, car, home) plus written SKILL instructions; can author its own connectors for services with APIs/CLIs
Agent	Self-extension	Builds its own tools and edits itself inside its sandbox
Agent	Web automation	Real Chromium browser; user can watch and take over
Agent	Payments	Wallet with single-use virtual cards (Stripe Link; Shop Pay announced)
Agent	Memory	Persistent memory about the user, stored as inspectable files in the VM

Known gaps per Meta: long-horizon agentic work and coding were still being improved at the Spark launch. Health reasoning was a stated focus, with training data curated with 1,000+ physicians.

Architecture 1: the Muse Spark model

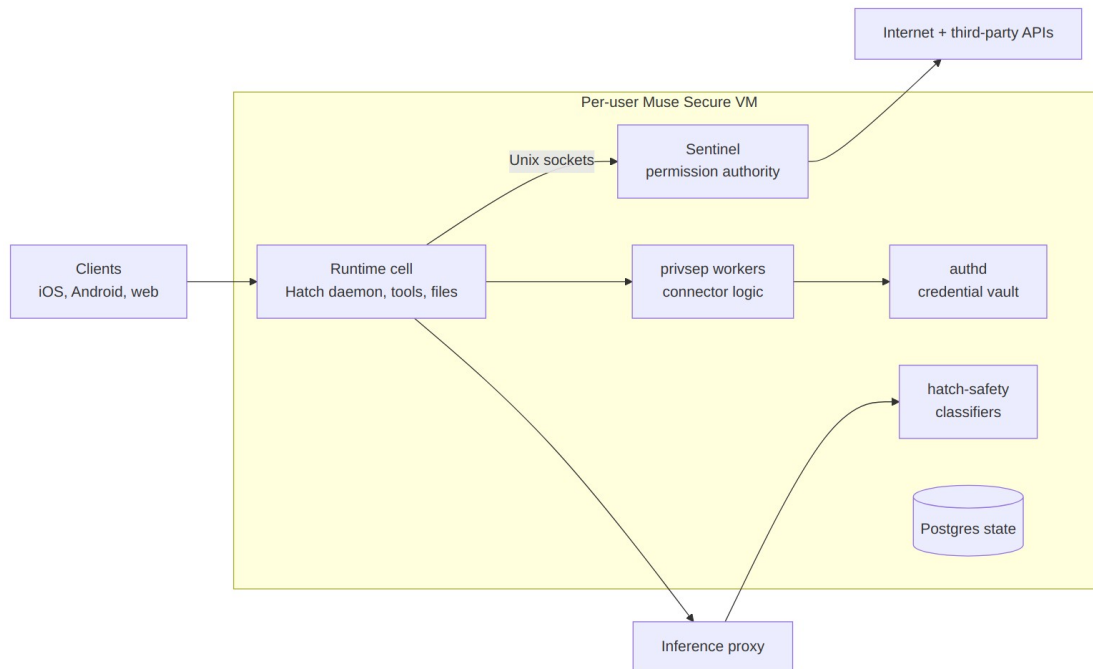
Muse Spark is built along three scaling axes: a rebuilt pre-training stack, large-scale RL, and efficient test-time reasoning. Meta has not published parameter counts or layer-level design.



- **Pre-training.** Rebuilt over nine months (architecture, optimizer, data curation). Meta reports reaching the same capability with over 10x less compute than Llama 4 Maverick, measured via scaling-law fits on small models.
- **Reinforcement learning.** Meta reports smooth, log-linear gains in pass@1 and pass@16 as RL steps scale, with gains generalizing to held-out tasks. For the agent, training emphasized tool calling via CLIs and skills, long-trajectory instruction following, prompt-injection awareness and multi-agent coordination.
- **Test-time reasoning.** RL maximizes correctness with a penalty on thinking time, producing “thought compression” (fewer reasoning tokens for the same result). To add reasoning without adding latency, Meta scales the number of parallel agents rather than the length of one chain.
- **Safety.** Evaluated under Meta’s Advanced AI Scaling Framework; pre-training data filtering, safety post-training and system guardrails. Apollo Research flagged unusually high evaluation awareness, which Meta judged non-blocking.
- **Access.** Hosted only: meta.ai, the Meta AI app, and the Meta Model API (muse-spark-1.1, US preview, \$1.25 / \$4.25 per million input / output tokens, OpenAI- and Anthropic-style request formats). No weights, no self-hosting, no fine-tuning.

Architecture 2: Muse Secure VM

Each user gets a dedicated Linux VM split into two security domains: an unprivileged runtime cell where the agent works on untrusted data, and host-side services that hold credentials, safety models and permissions. Meta’s framing: two isolated domains on one box, not an LLM agent with root.



The diagram shows the runtime cell reaching the outside world only through Sentinel, with credentials held outside the cell.

Component	Role	Isolation
Runtime cell	Runs the agent harness (“Hatch”), workspace files, binaries it executes	systemd - nspawn container; own Debian rootfs; cell root mapped to unprivileged host user; seccomp (no io_uring); no CAP_SYS_PTRACE or CAP_NET_ADMIN; virtual NIC
hatch-safety	Independent models/classifiers on every inference request and response (frontier risk, prompt injection)	Outside the cell, so an attacker in the cell cannot disable it
privsep workers	Execute built-in connector business logic	systemd-sandboxed; identified by cgroup; per-worker credential allowlist
hatch-authd	Stores OAuth tokens and passwords; mints surrogate tokens	Outside the cell; credentials stay in the user’s VM, not central Meta infra
Sentinel	Sole authority for connector actions and network egress	Host-side; agent cannot override
Postgres	Durable application state	Separate from both cell and

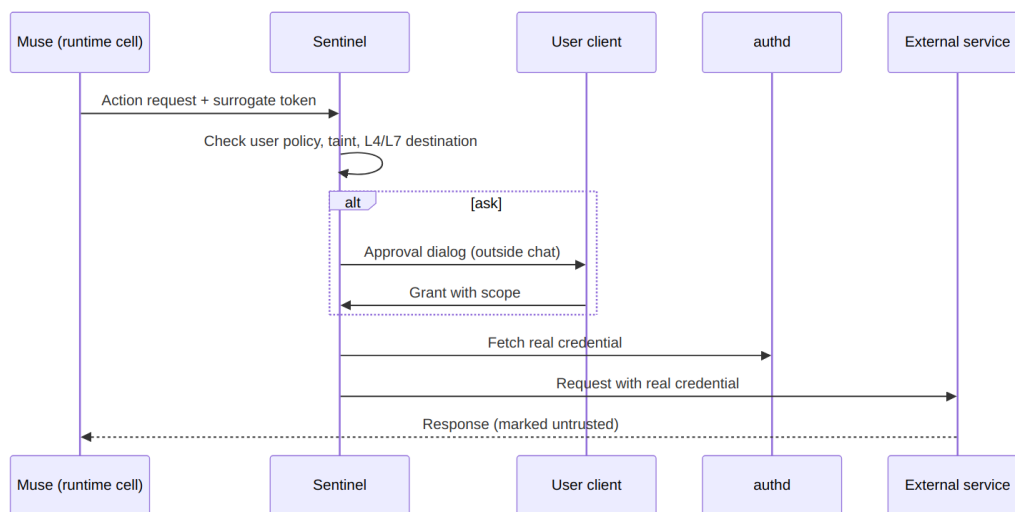
Component	Role	Isolation
Inference / telemetry proxies	Constrained paths to Meta infrastructure	credential store Only sanctioned exits besides Sentinel

All cell-to-service traffic uses Unix domain sockets with `SO_PEERCREDS` and peer ACLs, so callers are kernel-authenticated and there are no shared secrets to steal.

Connectors. Built-in connectors split in two: a thin CLI inside the cell only parses arguments and passes typed args and file descriptors over a socket; the credential-bearing logic runs in a `privsep` worker outside. A calendar worker cannot request an email credential by changing a parameter. The email connector also strips one-time passcodes, password-reset links and magic links using deterministic filters plus a classifier.

Architecture 3: Sentinel, credentials and human-in-the-loop

Muse proposes actions; only Sentinel can grant them. Every connector call and every network request is decided allow, deny or ask, and real secrets are injected only after that decision, at the network boundary.



The user's approval goes straight to Sentinel, never through the conversation, so the agent cannot fake or intercept it.

- **Connector policy.** Each request states connector, method, action class, scope and user context; Sentinel derives a user-visible purpose and applies the user's policy.
- **Egress control.** Traffic reaches Sentinel via a forward proxy built on user namespaces, veth boundaries and eBPF. Checks cover hostname, resolved and final IP, port, protocol, HTTP method, path and decoded body. SSRF checks stop public names resolving to private infrastructure.
- **Surrogate credentials.** Code in the cell only ever holds surrogate tokens minted by authd. Coercing the model to reveal a secret yields nothing usable.

- **Tainted egress.** Each tool process starts clean and becomes tainted once it reads user data, tracked in-kernel via eBPF cgroup programs and new LSM hooks. Clean requests under a narrow auto-allow policy pass silently; tainted ones fall back to approval.
- **Scoped grants.** Approvals are capabilities bound to a connector/destination and use case: one-time, session, task, time-bounded or perpetual. Sentinel enforces the exact scope on later calls.
- **Least privilege.** Read and write access are separated where the service allows, with finer controls than OAuth scopes (for example, Gmail read without Gmail settings).

Architecture 4: browser, payments, defense in depth, data

Muse layers probabilistic defenses (model training, classifiers) over deterministic ones (containers, credential ACLs, Sentinel), so a persuaded agent still hits hard boundaries.

Browser. A real Chromium runs behind a virtualization layer; a broker outside the cell owns the Chrome DevTools Protocol connection. The browser sub-agent sees an accessibility-tree snapshot, not raw DOM, and cannot run JavaScript, exec, or open devtools. Website logins are captured by a client UI, sent straight to authd, and injected at the point of need while the agent is paused. Extra classifiers watch for off-task personal-data egress, injection in DOM, images, media or downloads, and high-risk form submissions. Meta’s malicious-site blocklist is enforced in the VM.

Payments. Checkout pages are detected and every purchase needs an explicit approval showing exact details. For new merchants a wallet issues a single-use card number tied to one merchant, one amount and a short validity window.

Prompt-injection defense in depth. Meta frames the goal as breaking Simon Willison’s “lethal trifecta” (private data + untrusted content + exfiltration):

1. Model training to resist injection (Muse Spark 1.3 described as near state of the art).
2. Harness labels all external data as untrusted input.
3. An ensemble of independently trained injection classifiers on everything entering context.
4. Human approval for any action moving data out of the VM.

Bug bounty: up to \$300,000 per report, including up to \$130,000 for single-user prompt injection.

Data policy. The VM is the system of record; users can inspect, edit and download files and memory, with continuous backups. Conversations and VM data are not shared with ad systems, though the agent’s browsing looks like the user’s own activity to sites. Trajectories are PII-sanitized and used for training by default, with an opt-out. Meta staff access is currently limited by policy; a planned **Muse Confidential VM** aims to make it cryptographically impossible, with external audit.

Limitations

Muse is a consumer product on Meta’s cloud, so the main constraints for enterprise use are custody, availability and maturity, not raw capability.

- **No self-hosting or weights.** Model and agent run only on Meta infrastructure; no fine-tuning.
- **Geography.** Agent rollout and the Model API preview are US-only; the API is not open to EU developers.
- **Preview status.** No published SLAs or rate limits for the Model API at launch.
- **Operator access.** Meta can still access VM data to support, secure or operate the service until Confidential VM ships.
- **Prompt injection unsolved.** Meta states Muse will still make mistakes; its defenses bound impact rather than eliminate risk.
- **Lock-in.** Memory and context live in Meta’s ecosystem; the agent is tied to Muse Spark.

Part 2: Muse capabilities for a banking virtual assistant

Muse’s most transferable asset for banking is its security architecture: approval-gated actions, surrogate credentials and taint-aware egress map almost one-to-one onto how a bank must control an agent that can move money. The model capabilities are useful but interchangeable with other frontier models.

Mobile-only constraint. The bank’s VA lives only inside the native iOS and Android banking app. That shapes three things. First, the phone is a thin client: the agent runtime, browser and long-running tasks run server-side in the bank’s cloud, because iOS and Android suspend background work within seconds to minutes. Second, the app wakes the customer via push (APNs / FCM) and deep links for alerts and approvals. Third, approvals use native biometric sheets (Face ID / Android BiometricPrompt) bound to a registered device. Muse’s web client and WhatsApp surface do not apply.

Model capabilities

Muse capability	Banking VA use case (mobile)	Fit
Tool calling (parallel, JSON schema)	Balance and transaction lookups, card freeze/unfreeze, bill pay, dispute filing via core-banking and card APIs	High
Multimodal input (images, PDFs)	Camera capture of a bill, cheque or receipt; parse statements, payslips or KYC documents from the photo library or Files; explain a screenshot	High (camera is the phone’s strongest input)
Audio input	Voice requests (“freeze my card”) for one-handed use and accessibility	High
Long context (1M tokens)	Reason over months of transactions, product T&Cs and fee schedules, or a long	High

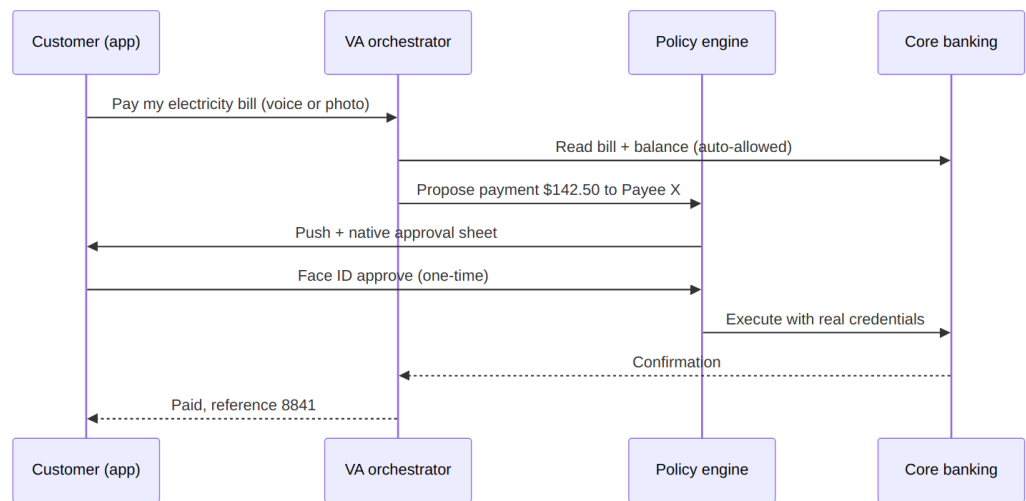
Muse capability	Banking VA use case (mobile)	Fit
	complaint history in one server-side pass	
Reasoning / Contemplating mode	Spend analysis, budgeting, loan affordability explanations	Medium (deterministic calculators for final numbers; parallel mode adds latency on mobile)
Prompt-injection resistance	Safely read merchant descriptors, uploaded documents and web pages that may carry injected text	High
Visual chain-of-thought	Annotate a screenshot of the app to guide the customer	Low to medium (small screen; native UI cards usually beat annotated images)

Agent capabilities

Muse capability	Banking VA use case (mobile)	Fit
Sentinel allow / deny / ask	Server-side policy engine for every banking action: reads auto-allowed, transfers and payee changes require step-up approval	High
Out-of-band approval dialog	Native bottom sheet outside the chat thread, showing exact amount and payee, confirmed with Face ID / BiometricPrompt and device-bound keys (SCA)	High
Push-routed approvals	When a background task needs consent, push notification deep-links to the approval sheet; approval expires if unanswered	High (mobile-specific adaptation)
Scoped grants (one-time, task, time-bounded)	Mandates such as “pay this biller up to \$200 monthly for 6 months” without re-prompting	High
Surrogate credentials + vault	LLM never sees account numbers, OAuth tokens or open-banking consents; real values swapped in at the API	High

Muse capability	Banking VA use case (mobile)	Fit
Tainted egress	gateway Any process that has read customer PII loses auto-allow, blocking silent exfiltration	High
Single-use virtual cards	Agent-initiated purchases and subscriptions paid with merchant-locked, amount-locked virtual cards	High (bank can issue these natively)
OTP / reset-link filtering	Strip OTPs, reset links and security codes from anything the agent reads	High
Read/write separation	Launch read-only (insights, Q&A) first, graduate to write actions per customer consent	High
Background tasks + crons	Server-side workflows raise proactive alerts via push: low balance, unusual spend, bill due, subscription price rise	High (must run server-side; the app cannot host them)
Sub-agents	Specialist agents for disputes, loans, card services, fraud triage under one orchestrator	Medium
Browser sub-agent	Cancel subscriptions, retrieve bills from biller portals; runs in a bank-hosted cloud browser, streamed to the app for takeover	Medium (see “Browser flows in the mobile VA”)
Persistent, inspectable memory	Remember goals and preferences, with an in-app screen to view and delete them	Medium
Messaging surfaces (WhatsApp, web)	Not applicable: the VA is app-only	N/A
Self-authored connectors	Agent writing its own integrations	Low (unacceptable under bank change-management controls)

Reference flow for a payment



The VA only proposes; the policy engine, not the model, holds the authority and the credentials.

Adoption considerations for a bank

The practical path is to borrow Muse’s architecture patterns and treat the model as one swappable option; the consumer Muse agent itself is not something a bank can deploy.

Option	What it means	Viability today
Use the Muse consumer agent	Customers link their bank to Muse	Not a bank product; Meta holds custody; no enterprise controls
Muse Spark via Meta Model API	Bank builds its own VA with Muse Spark as the LLM	Possible for US pilots; preview status, no SLAs, no EU access, customer data leaves bank perimeter
Replicate the Muse architecture	Bank builds VM/container isolation, policy engine, credential vault and approvals, with any approved LLM	Recommended; patterns are published and model-agnostic

Regulatory and risk points to settle before any build:

- **Data residency and third-party risk.** Cloud-only hosting outside the bank’s perimeter triggers outsourcing and vendor-risk review (e.g. DORA in the EU, OCC third-party guidance in the US).
- **Model risk management.** Closed weights and no fine-tuning limit validation under SR 11-7-style frameworks; keep all money calculations deterministic.
- **Strong customer authentication.** Approval dialogs must be tied to the bank’s SCA / step-up auth, not just a tap.

- **Explainability and audit.** Log every proposal, policy decision and approval as an immutable audit trail; Sentinel's decision records are a good template.
- **Training data.** Muse's default of training on sanitized trajectories must be disabled or contractually excluded for banking data.
- **Fair lending and advice.** Product recommendations or credit explanations fall under conduct rules; scope the VA to information and execution unless advice is licensed.

Mobile-specific points:

- **Thin client.** No model, agent logic or secrets on the device; the app renders chat, native cards and approval sheets, and holds only a device-bound signing key in Secure Enclave / StrongBox.
- **Durable tasks.** Anything longer than one request runs as a server-side workflow that survives the app being closed; state syncs back on reopen.
- **Consent UX.** Approval sheets must fit one screen: action, amount, counterparty, scope, expiry. Never ask for approval inside the chat bubble.
- **App-store and OS rules.** Background refresh cannot be relied on; push is the only reliable wake-up. Embedded browser views must follow Apple and Google WebView policies.
- **Network variance.** Stream responses, keep payloads small, and make every action idempotent so a retry on a flaky connection never pays twice.

Three Muse-style use cases for the mobile VA

Each use case below copies Muse's core loop. The agent plans and acts server-side, the customer approves consequential steps on the phone, and the agent keeps working after the app closes.

1. Subscription manager: find and cancel

Customer says: "What am I still paying for? Cancel the ones I don't use."

1. The agent scans 12 months of card and account transactions and clusters recurring merchants. It shows a native list card with each merchant, the monthly amount, the last price change and the next charge date.
2. The customer taps the subscriptions to cancel.
3. For each one, the agent picks the cheapest reliable route, in order: merchant API, a cloud-browser cancellation flow, then a guided in-app walkthrough. If all of those fail, it blocks the merchant's recurring charges at the card level.
4. Before the final "confirm cancellation" step, the customer gets a push and a Face ID approval sheet.
5. The agent saves proof (a screenshot of the confirmation page and a timestamp). It then watches for the next expected charge. If the merchant bills anyway, it pushes a pre-filled dispute.

Muse patterns used: background monitoring, browser sub-agent, human-in-the-loop on high-risk form submission, credential vault for merchant logins.

2. Snap-and-pay bills with standing mandates

Customer says: “Pay this” (with a photo of a utility bill), then “and handle it every month.”

1. Multimodal parsing extracts the biller, account reference, amount and due date. The agent matches the biller to an existing payee or proposes adding a new one; a new payee always needs step-up approval.
2. The customer approves a one-time payment with Face ID.
3. On “every month”, the policy engine offers a scoped grant, for example: “up to \$200 per month to this biller for 12 months, auto-approved within the limit.”
4. A monthly workflow fetches the new bill from the biller API or portal and pays it within the mandate. A bill over the limit, or from a new biller account, falls back to a push approval.

Muse patterns used: multimodal input, scoped time-bounded grants, crons, allow / deny / ask policy.

3. Dispute and chargeback agent

Customer says: “I was charged twice at Hotel X” or “my order never arrived.”

1. A dispute sub-agent finds the matching transactions and checks whether they are pending or posted. It also checks the merchant’s refund history.
2. It asks for evidence in the chat: photos of receipts, screenshots of the order, or delivery notes. It classifies the case against card-scheme reason codes.
3. It drafts the dispute and offers a provisional credit where policy allows. The customer approves submission in a native sheet.
4. A background workflow tracks the case through the scheme timeline. It pushes status updates and deadlines, and asks for more evidence when the merchant responds.

Muse patterns used: sub-agent specialisation, long context over transaction history, multimodal evidence, long-running background tasks, push-routed follow-ups.

Use case	Build effort	Main dependency	Suggested order
Snap-and-pay bills	Low to medium	Biller directory / payee APIs	1st (mostly internal APIs)
Dispute agent	Medium	Card-dispute platform APIs	2nd
Subscription manager	Medium to high	Merchant APIs + cloud browser	3rd (browser flows are the riskiest part)

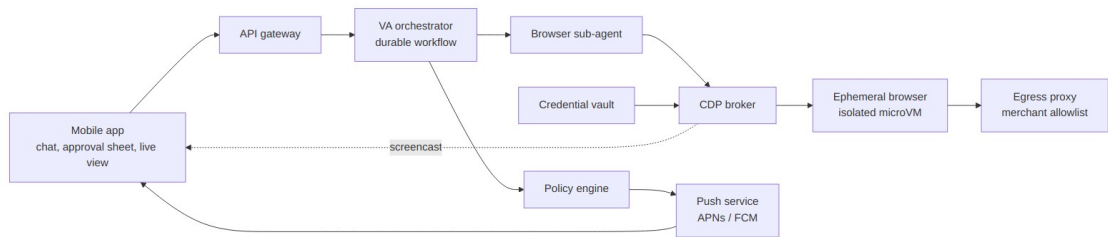
Browser flows in the mobile VA (cancel a subscription)

Run the browser in the bank’s cloud, not on the phone. Stream it into the app only when the customer needs to take over (login, 2FA, CAPTCHA). This mirrors Muse’s design and fits mobile-only limits: the phone can’t host a long-running automated browser, and iOS suspends apps in the background.

Integration options, in fallback order

Route	How it works	Use when
1. Merchant / aggregator API	Cancel through a partner API or a subscription-management provider	Merchant supports it; most reliable, no browser
2. Cloud browser agent	Server-side Chromium driven by the browser sub-agent, streamed to the app for takeover	No API, and the merchant allows online cancellation
3. Guided in-app WebView	Merchant’s cancel page opens in an in-app browser; the agent gives step-by-step guidance in a native overlay; the customer taps through	Bot defences block automation, or the customer prefers to do it themselves
4. Card-level block	Stop future recurring charges from that merchant, or reissue a virtual card number	Every other route fails; it doesn’t cancel the contract, so tell the customer

Architecture



The agent never touches the browser directly; the broker mediates every action and injects merchant credentials from the vault.

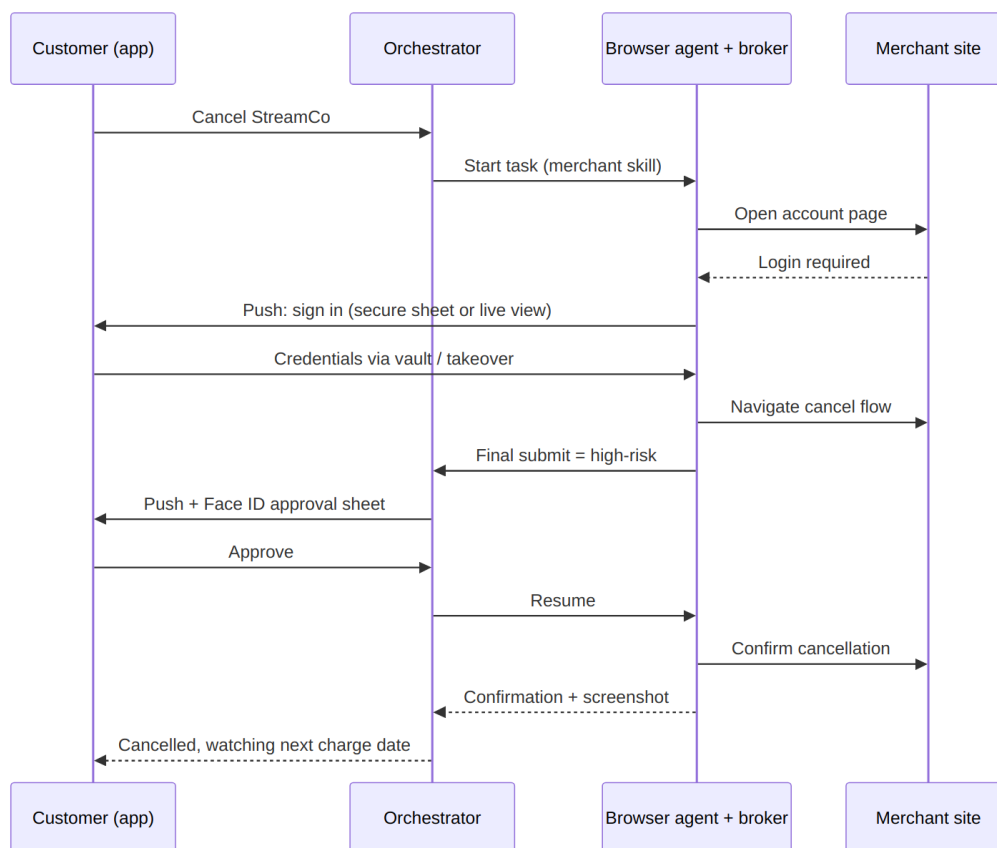
Key components

- **Ephemeral browser pool.** One headless Chromium (driven through Playwright or CDP) per task, inside a Firecracker or gVisor sandbox, destroyed after the task. Cookies persist only in an encrypted, per-customer profile if the customer opts in.
- **CDP broker.** Muse’s pattern: the sub-agent gets an accessibility-tree snapshot and a small verb set (click, type, select, scroll). It gets no JavaScript execution and no raw DOM. The broker enforces a per-merchant domain allowlist and blocks file downloads.
- **Browser sub-agent.** Guided by per-merchant “skills”: known cancellation paths for the top merchants, kept as data. It falls back to generic navigation for the long tail. All page content is labelled untrusted and passed through injection classifiers.
- **Credential vault.** Merchant logins are entered in a native secure sheet in the app and sent straight to the vault. The broker fills them into the page while the agent is paused, so the LLM never sees them.
- **Live view and takeover.** The broker streams frames (CDP screencast over WebSocket, or WebRTC for smoother video) into a native view in the app. Touches are mapped back

to browser input. The agent pauses during takeover, and the customer clears 2FA or CAPTCHA and hands back.

- **High-risk step gate.** The policy engine marks the final “Cancel subscription” submit as ask. The workflow pauses and a push opens a native sheet showing the merchant, the plan and the page screenshot. Face ID approval resumes the task.
- **Durable workflow.** A workflow engine such as Temporal holds task state across the push round-trip, takeovers and retries. Timeouts end in a clear “not completed” state, never a silent half-done cancellation.
- **Evidence and audit.** Save a final-page screenshot, the confirmation number and a DOM hash. Link them to the transaction for later disputes. Log every agent action and approval immutably.

Runtime sequence



Risks to plan for

- **Bot detection and ToS.** Merchants may block data-centre IPs or forbid automation. Keep merchant partnerships, respect robots and ToS, and fall back to route 3 rather than evading defences.
- **Retention dark patterns.** Offers and multi-step “are you sure” screens need merchant-specific skills and a rule that the agent never accepts an offer without asking.
- **Liability and consent.** Get explicit per-merchant consent to act on the customer’s behalf, and show the customer exactly what was done.

- **Mobile UX.** Keep the live view optional; most customers should only see two taps: pick the subscription, approve the final step.

Sources

- [How We Built Safety Into Muse](#), Meta AI Research, Sep 8, 2026 (primary source for agent architecture)
- [Introducing Muse Spark](#), Meta AI, Apr 8, 2026
- [Official Muse Breakdown](#), Vellum, Sep 9, 2026 (competitor; used for context only)
- [Meta Launches Muse Spark 1.1 With Its First Paid AI Model API](#)
- [Meta Muse Spark 1.1 developer guide](#)
- [Muse \(Learn AI wiki\)](#), for Muse Image and naming